Итоговый тест.

Полученные знания всегда нужно применять на практике, поэтому мы сначала повторим теорию, а затем перейдем к практике.

Оглавление:

1. Теория
2. Задание №1
3. Задание №2
4. Задание №3
5. Задание №4

Теория

Целочисленные типы данных используются для хранения целых чисел. Не забывайте о проблемах деления и переполнения с ними. Используйте целочисленные типы с фиксированным размером.

Типы данных с плавающей точкой используются для хранения вещественных чисел. Не забывайте о проблемах с точностью, ошибках округления и неточном сравнении чисел.

Логический тип данных содержит 2 значения: `true` и `false`.

Символьный тип данных содержит целые числа, которые могут интерпретироваться в символы, соответствующие стандарту ASCII. Будьте осторожны при использовании фактических чисел и цифр, которые используются для представления символов. Также помните о проблемах переполнения.

Используйте **спецификатор const** для объявления символьных констант вместо использования директив #define. Это безопаснее.

Задание №1

Почему символьные константы лучше литеральных (магических чисел)? Почему использование `const` лучше использования директив #define?

Ответ №1

Использование литеральных констант (магических чисел) не только усложняет программу, но и затрудняет внесение в нее изменений. Символьные константы лучше, так как они дают понимание того, зачем и почему они используются, а также, если вам еще понадобится внести изменения — достаточно будет внести правки только в объявлении константы, а не искать их по всему коду. Значения констант, объявленных с помощью директивы #define, не отображаются в отладчике, вероятность возникновения конфликта имен у таких констант выше.

Задание №2

Выберите подходящий тип данных для переменных в каждой из следующих ситуаций. Будьте как можно более конкретными. Если ответом является целочисленный тип данных, то используйте соответствующий тип с фиксированным размером (например, int16_t). Если переменная должна быть константной, то так и отвечайте.

- Возраст пользователя.
- Нравится ли определенный цвет пользователю?
- Число Пи.
- Количество страниц в учебнике.
- Цена акций в долларах (дробь присутствует).
- Сколько раз вы моргнули за всю свою жизнь? (*Примечание:* Ответ исчисляется в миллионах)
- Пользователь выбирает опцию с помощью ввода определенной буквы.

Ответ №2

- int8_t не сможет хранить возраст человека, старше 127. Несмотря на то, что таких случаев в мире единицы (если есть вообще), в будущем это может быть вполне возможным. Так что int16_t здесь подойдет лучше.
- bool.
- const double.
- Так как большинство книг имеют больше 255 страниц, но меньше чем 32767, то int16_t — здесь наилучший вариант.
- float.
- int32_t.
- char.

Задание №3

Напишите следующую программу. Сначала пользователю предлагается ввести 2 числа типа с плавающей точкой (используйте тип double). Затем предлагается ввести один из следующих математических символов: `+`, `-`, `*` или `/`. Программа выполняет выбранную пользователем математическую операцию между двумя числами, а затем выводит результат на экран. Если пользователь ввел некорректный символ, то программа ничего не должна выводить. Например:

```
Enter a double value: 7
Enter a double value: 5
Enter one of the following: +, -, *, or /: *
7 * 5 = 35
```

> ***Пейсказка:*** Вы можете использовать ветвление if для того, чтобы распознать, ввел ли пользователь определенный математический символ (например, `+`) или нет.

Ответ №3

```
#include <iostream>

double getDouble()
{
    std::cout << "Enter a double value: ";
    double x;
    std::cin >> x;
    return x;
}

char getOperator()
{
    std::cout << "Enter one of the following: +, -, *, or / ";
    char op;
    std::cin >> op;
    return op;
}

void printResult(double x, char op, double y)
{
    if (op == '+')
        std::cout << x << " + " << y << " = " << x + y << '\n';
    else if (op == '-')
        std::cout << x << " - " << y << " = " << x - y << '\n';
    else if (op == '*')
        std::cout << x << " * " << y << " = " << x * y << '\n';
    else if (op == '/')
        std::cout << x << " / " << y << " = " << x / y << '\n';
}

int main()
{
    double x = getDouble();
    double y = getDouble();

    char op = getOperator();

    printResult(x, op, y);

    return 0;
}
```

Задание №4

Это уже немного сложнее. Напишите небольшую программу-симулятор падения мячика с башни. Сначала пользователю предлагается ввести высоту башни в метрах. Не забывайте о гравитации (`9.8м/с²`) и о том, что у мячика нет начальной скорости (его держат в руках). Программа должна выводить расстояние от земли, на котором находится мячик после 0, 1, 2, 3, 4 и 5 секунд падения. Минимальная высота составляет 0 метров (ниже мячику падать нельзя).

В вашей программе должен быть заголовочный файл constants.h с пространством имен `myConstants`. В `myConstants` определите константу для хранения значения силы тяжести на Земле (`9.8`).

Напишите функцию для вычисления высоты мячика через `x` секунд падения. Используйте следующую формулу: *высота мячика над землей = константа_гравитации * x_секунд²/2*.

Пример результата выполнения программы:

```
Enter the initial height of the tower in meters: 100
At 0 seconds, the ball is at height: 100 meters
At 1 seconds, the ball is at height: 95.1 meters
At 2 seconds, the ball is at height: 80.4 meters
At 3 seconds, the ball is at height: 55.9 meters
At 4 seconds, the ball is at height: 21.6 meters
At 5 seconds, the ball is on the ground.
```

> ***Примечания:***
>
> - В зависимости от начальной высоты, мячик может и не достичь земли в течение 5 секунд — это нормально. Мы усовершенствуем эту программу, когда будем рассматривать циклы.
> - Символ `^` не является экспонентом в языке C++. В формуле вместо него используйте знак умножения `*`.

Ответ №4

Файл constants.h:

```
#ifndef CONSTANTS_H
#define CONSTANTS_H

namespace myConstants
{
    const double gravity(9.8);
}
#endif
```

Файл main.cpp:

```
#include <iostream>
#include "constants.h"

// Получаем начальную высоту от пользователя и возвращаем её
double getInitialHeight()
{
    std::cout << "Enter the initial height of the tower in meters: ";
    double initialHeight;
    std::cin >> initialHeight;
    return initialHeight;
}

// Возвращаем расстояние от земли после "..." секунд падения
double calculateHeight(double initialHeight, int seconds)
{
    // Используем формулу: [ s = u * t + (a * t*2) / 2 ], где u(начальная скорость) = 0
    double distanceFallen = (myConstants::gravity * seconds * seconds) / 2;
    double currentHeight = initialHeight - distanceFallen;

    return currentHeight;
}

// Выводим высоту, на которой находится мячик после каждой секунды падения
void printHeight(double height, int seconds)
{
    if (height > 0.0)
        std::cout << "At " << seconds << " seconds, the ball is at height: " << height << " meters\n";
    else
        std::cout << "At " << seconds << " seconds, the ball is on the ground.\n";
}

void calculateAndPrintHeight(double initialHeight, int seconds)
{
    double height = calculateHeight(initialHeight, seconds);
    printHeight(height, seconds);
}

int main()
{
    const double initialHeight = getInitialHeight();

    calculateAndPrintHeight(initialHeight, 0);
    calculateAndPrintHeight(initialHeight, 1);
    calculateAndPrintHeight(initialHeight, 2);
    calculateAndPrintHeight(initialHeight, 3);
    calculateAndPrintHeight(initialHeight, 4);
    calculateAndPrintHeight(initialHeight, 5);

    return 0;
}
```